

DA2304: Introduction to Computer Systems

Assignment 3 — Virtual Machines & Jack Compiler

Venkata Sreekar

May 2026

Part 1

Exercise 1 — 2D Convolution

I chose **stride** $S = 1$. This preserves full spatial resolution, producing a 3×3 output from the 5×5 input — consistent with Figure 1 in the assignment specification. Stride 1 is appropriate here because the input is small (5×5); using stride 2 would reduce the output to a 2×2 matrix, discarding most of the spatial information. In production CNNs, stride 2 is preferred for downsampling, but for this demonstration stride 1 yields a richer and more verifiable output.

The output dimension is computed via:

$$O = \left\lfloor \frac{N - F + 2P}{S} \right\rfloor + 1 = \left\lfloor \frac{5 - 3 + 0}{1} \right\rfloor + 1 = 3$$

Exercise 2 — The Conv Class in Jack

The convolution class is implemented in `Conv.jack` (located in the `jack/` folder). The driver program using a 5×5 input matrix is implemented in `Main.jack` in the same folder.

Part 2

Exercise 1 — Tokenizer

The tokenizer is implemented in `JackTokenizer.py` (Python), located in the `src/` folder. It converts a `.jack` source file into a token-stream XML file (`<ClassName>T.xml`).

Exercise 2 — The Parser (Syntax Analyzer + VM Code Generator)

1. Sections 2.1 and 2.2 (parse-tree XML and VM code generation) are implemented in `CompilationEngine.py`, located in the `src/` folder.
2. Section 2.3 (end-to-end pipeline) is implemented in `JackCompiler.py`, also in the `src/` folder.

Exercise 3 — Analysis Questions

3.1. Tokenizer Design

Question: Why is it important that the tokenizer handles `/* ... */` comments using a state machine rather than a simple regex on a single line? Give a concrete failure case.

A single-line regex such as `/\*.?*\/` only matches comment spans that begin and end *on the same line*. This fails silently for multi-line block comments — a common and entirely valid construct in Jack.

Concrete failure case. Consider the following excerpt:

```

/** Compute output dimension using
    the formula 0 = floor(...) + 1. */
method int calcOutputDim() { ... }

```

A line-by-line regex would see `/** Compute output dimension using` and find no closing `*/` on that line; the comment is therefore left open. The next line (`the formula...`) is then passed to the parser as source code, producing a tokenisation error. Worse, the closing `*/` on line 3 may itself be misparsed, causing the real code on line 4 to be silently swallowed into the “comment.”

A state machine solves this by maintaining a boolean flag `in_block_comment`. On encountering `/*` (or `/**`), the flag is set; every subsequent character is consumed without emission until `*/` is found and the flag is cleared. This works correctly regardless of how many newlines the comment spans.

3.2. Symbol Table Walk-through

Question: Walk through the symbol-table state when the parser enters `Conv.convolve()`. Show the kind, type, and index for every identifier in scope at the point where the inner-loop accumulator is assigned.

At the point where `sum = sum + ...` is executed inside the innermost loop of `conv2d()`, two symbol-table scopes are active simultaneously.

Class-level scope (populated when the class declaration is compiled):

Identifier	Kind	Type	Index
<code>ArrayFilter</code>	static	Array	0
<code>stride</code>	static	int	1
<code>ArrayInput</code>	field	Array	0
<code>ArrayOutput</code>	field	Array	1
<code>ArraySize</code>	field	int	2

Subroutine-level scope (populated on entry to `conv2d()`, a method):

Identifier	Kind	Type	Index
<code>this</code>	argument	Conv	0
<code>outDim</code>	local	int	0
<code>i</code>	local	int	1
<code>j</code>	local	int	2
<code>fr</code>	local	int	3
<code>fc</code>	local	int	4
<code>sum</code>	local	int	5
<code>oi</code>	local	int	6
<code>oj</code>	local	int	7
<code>k</code>	local	int	8
<code>inputIdx</code>	local	int	9

Memory mapping: static variables map to the `static` segment; field variables map to the `this` segment (accessed via `pointer 0`); subroutine local variables map to the `local` segment; and the implicit `this` argument of a method occupies `argument 0`.

3.3. Compilation Correctness — Tracing `let ArrayOutput[k] = sum;`

Question: For your 5×5 test case, manually trace the VM code emitted for `let ArrayOutput[k] = sum;` and explain how the `pointer 1 / that 0` idiom resolves to the correct RAM address.

The Jack compiler translates an array-write `let arr[index] = expr;` into the following four-step VM sequence.

Step 1 — Compute the target address. Push the base address of `ArrayOutput` (a field variable at `this 1`) and the index `k` (a local variable at `local 8`), then add them:

```
push this 1    // base address of ArrayOutput
push local 8    // value of k
add            // base + k → target RAM address on stack
```

Step 2 — Evaluate the right-hand side.

```
push local 5    // value of sum
```

Step 3 — Store sum in a temporary register. Because the VM has only one stack, the computed target address must be kept alive while the RHS is placed. The standard idiom saves `sum` to `temp 0`:

```
pop temp 0      // save sum temporarily
```

Step 4 — Install the target address and write. Pop the target address into `pointer 1`, which sets the `THAT` base register to that RAM address. Then push `sum` back and pop it to `that 0`:

```
pop pointer 1    // THAT = base + k (i.e. &ArrayOutput[k])
push temp 0      // restore sum
pop that 0       // RAM[THAT + 0] = sum → ArrayOutput[k] = sum
```

Why this works: In the Hack VM, `pointer 1` directly sets register `R4 (THAT)`. A subsequent `that 0` accesses `RAM[THAT]`, which is exactly `RAM[base_ArrayOutput + k]` — the correct element of the output array.

3.4. Convolution Stride Analysis

Question: For a 9×9 input with $F = 3$ and $S = 2$, what is the output dimension? Do $S = 1$ and $S = 2$ change the asymptotic VM-instruction count of `convolve()`? Explain.

Output dimension ($N = 9, F = 3, P = 0, S = 2$):

$$O = \left\lfloor \frac{9 - 3 + 0}{2} \right\rfloor + 1 = \lfloor 3 \rfloor + 1 = 4$$

The output is a 4×4 matrix (16 elements).

VM instruction count. The body of `conv2d()` contains three nested loops:

- i. Outer loops iterate over output positions: $O \times O$ times.
- ii. Inner loops iterate over the filter: $F \times F = 9$ times (fixed, independent of S).

The number of multiply-accumulate (MAC) operations is therefore $O^2 \times F^2$. Substituting:

$$\text{MACs}(S) = O(S)^2 \times 9 \quad \text{where} \quad O(S) = \left\lfloor \frac{N - F}{S} \right\rfloor + 1$$

For a fixed input size N :

S	O	Total MACs
1	$N - 2$	$9(N - 2)^2$
2	$\lfloor (N - 2)/2 \rfloor + 1$	$\approx 9 \left(\frac{N}{2}\right)^2$

Asymptotically, both are $\Theta(N^2)$ — the inner 3×3 filter work is $O(1)$ with respect to N . However, $S = 2$ reduces the *constant factor* by approximately $4 \times$ compared to $S = 1$, since the outer loops execute roughly $\frac{1}{4}$ as many iterations. In terms of actual VM instructions emitted for `conv2d()`, the loop structure (push, pop, add, call) is identical in form for both strides; only the runtime iteration count differs. Therefore the **asymptotic instruction count is the same**, $\Theta(N^2)$, but stride 2 is roughly $4 \times$ faster in practice for large N .

Conv Inputs and Filter

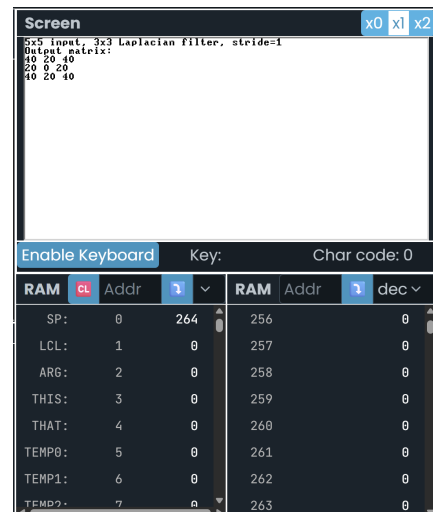
Laplacian Edge-Detector Filter (3×3):

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

1. 5×5 Input Matrix

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 20 & 20 & 20 & 0 \\ 0 & 20 & 20 & 20 & 0 \\ 0 & 20 & 20 & 20 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Input: 5×5 Block

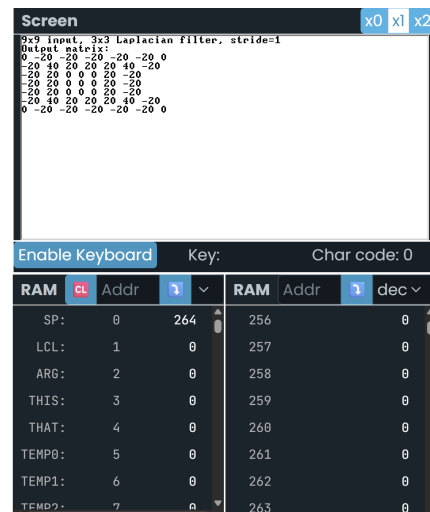


Output: 3×3 Result

2. 9×9 Input Matrix

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 20 & 20 & 20 & 20 & 20 & 0 & 0 \\ 0 & 0 & 20 & 20 & 20 & 20 & 20 & 0 & 0 \\ 0 & 0 & 20 & 20 & 20 & 20 & 20 & 0 & 0 \\ 0 & 0 & 20 & 20 & 20 & 20 & 20 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Input: 9x9 Block



Output: 7x7 Result

Extra Credit

While building the parser, in a recursive-descent parser, methods (like `compile_statement` or `compile_expression`) rely on a shared cursor pointing to the `current_token`.

- **The Error:** A method consumes a token it was supposed to just look at, or it forgets to advance past a token it finished processing.
- **The Result:** The entire token stream shifts. The `compile_while` method might suddenly read an opening bracket `"{"` instead of the word `while`, causing a massive cascade of syntax errors that are incredibly hard to trace back to the original source.
- **The Fix:** Establish a strict, ironclad rule for your methods. For example: *Every `compile_x` method must assume the cursor is already sitting on its first token when it starts, and it must advance the cursor exactly past its last token before it returns.*